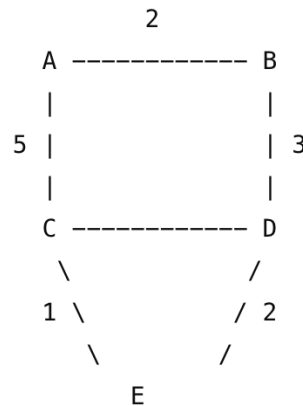


Exercise 2: Link State and Distance Vector Routing

Name: Nathakorn Fanrahan Student ID: 6710450716 Date:

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	inf	inf
1	B	2, A	5, A	5, B	inf
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:

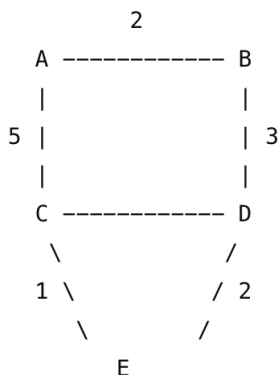
Destination	Shortest Path	Total Cost
B	A->B	2
C	A->C	5
D	A->B->D	5
E	A->C->E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A->B
C	5	C	A->C
D	5	B	A->B->D
E	6	C	A->C->E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2+0 = 2$	$4+0 = 4$	2	B
C	$2+4 = 6$	$5+0 = 5$	5	C
D	$2+3 = 5$	$5+1 = 6$	5	B
E	$2+5 = 7$	$5+1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A->B
C	C	5	A->C
D	B	5	A->B->D
E	C	6	A->C->E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

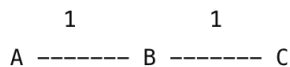
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
โหนดที่ต่อสายตรงอยู่ คือ C และ D จะรู้ตัวทันที ผ่านการส่ง Hello Packet หากันแล้ว Timeout	โหนด C และ D ตรวจพบว่าลิงก์ขาด จึงเปลี่ยนค่า Cost ในการไปยังปลายทางนั้นเป็น อนันต์ ในตารางของตัวเอง
โหนด C และ D จะสร้างแพ็กเก็ตอัปเดตหรือ LSP จะแจ้งเฉพาะข้อมูลสั้นๆ ว่าสายที่ต่อระหว่างอินกับ C/D ขาด แล้วส่งกระจาย ไปยังโหนด ทั้งหมด ในเครือข่ายทันที	C และ D จะส่งตารางที่อัปเดตใหม่นี้ไปให้ เฉพาะเพื่อนบ้านที่ต่อตรงเท่านั้น แล้วเพื่อนบ้านก็ค่อยๆ ส่งต่อไปเป็นทอดๆ
โหนดอื่นๆ เมื่อได้รับข้อความ จะเอาไปแก้ไข แผนที่เครือข่ายหรือ Topology Database ของตัวเองว่าตรงนี้ไม่มีทางไปแล้ว	ระหว่างที่ข่าวสารค่อยๆ กระจาย โหนดที่อยู่ไกลๆ อาจยังถือข้อมูลแผนที่เก่าอยู่ ทำให้หลงคิดว่ายังมีทางผ่าน C ไป D ได้ จึงยังคงส่งข้อมูลไปทางเดิม ทำให้แพ็กเก็ตวิ่งวนหรือสูญหาย
ทุกโหนดจะนำแผนที่ใหม่มารัน algorithm Dijkstra ใหม่ทั้งหมดด้วยตัวเอง เพื่ออัปเดตตาราง Forwarding	จะสำเร็จก็ต่อเมื่อการอัปเดตเป็นทอดๆหรือบวกค่าเพิ่มไปเรื่อยๆ เสร็จสมบูรณ์ทุกโหนด ซึ่งช้ากว่า LS มาก

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

- เมื่อ B-C ขาด B ไม่รู้จะไป C ทางไหน แต่ B ดันได้ตารางข้อมูลเก่าจาก A ทำให้ B โยนแพ็กเก็ตกลับไปหา A แต่ A ก็โยนกลับมาหา B เพราะ A คิดว่าทางไป C ต้องผ่าน B เกิดเป็นลูป

2. Show how the estimated distance to C can increase over successive iterations.

- B อัปเดตตารางตัวเอง เสียค่าไป A(1) + ค่าที่ A บอกว่าไป C(2) จะได้เป็น $1+2 = 3$ แล้วส่งตารางบอก A

- A อัปเดตตารางตัวเอง คำนวณใหม่ เสียค่าไป B (1) + ค่าที่ B(3) จะได้เป็น $1+3 = 4$ แล้วส่งกลับไปที่บอก B

- B อัปเดตอีกรอบ เป็น $1+4 = 5$ ซึ่งค่าจะทบไปเรื่อยๆ

3. What is count-to-infinity?

- เป็นค่าที่ Router หาค่าไม่เจอจนค่า cost วิ่งหาเรื่อยๆจนแตะเพดานค่า inf ที่ protocol กำหนดไว้

4. Why does it occur in Distance Vector routing?

- DV ทำงานแบบ **Routing by Rumor** เราเตอร์ไม่เห็นแผนที่ทั้งหมดขึ้นยืนด้วยตาตัวเอง รู้แค่เพื่อนบ้านบอกมาแบบไหนก็บอกเลขตามนั้น

5. Name a mitigation approach.

- ให้งู **Split Horizon** รู้เส้นทางไหนมาจากใคร ห้ามส่งข้อมูลเส้นทางนั้นกลับไปที่หาคนนั้น เช่น A เรียนทางไป C มาจาก B A ก็จะไม่ส่งอัปเดตเรื่อง C กลับไปให้ B

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra	Bellman-Ford
Information maintained	Linkstate Database	Distance Table
Information exchanged	สถานะของสายที่ต่อตรงกับตัวเอง (LSP)	ตาราง Routing ทั้งหมดของตัวเอง
Algorithm	Global	Decentralized
Topology knowledge	รู้ครบทั้งกระดาน	รู้แค่เพื่อนบ้านตัวเอง
Routing updates	ส่งเฉพาะเวลาที่เกิดการเปลี่ยนแปลง	ส่งตามรอบเวลาหรือเมื่อเปลี่ยน
Convergence	เร็ว	ช้า
Routing loops	อาจเกิดเป็นวงวนชั่วคราว	เกิดขึ้นบ่อย
Count-to-infinity	ไม่มีปัญหา	มีปัญหา
Example protocol	OSPF, IS-IS	RIP, BGP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

- ใช้ **Link State** เพราะต้องการความไว เมื่อระบบล่ม **LS** จะแจ้งเตือนทันทีด้วยการ **Flood** ข้อมูล ทำให้เครือข่ายกลับมาทำงานปกติได้รวดเร็ว ลด **Downtime**
- โหนดมีมุมมองเครือข่ายที่สมบูรณ์ ป้องกันปัญหา **Count-to-infinity** และคำนวณเส้นทางใหม่ได้อย่างแม่นยำ
- โปรโตคอลสาย **LS** อย่าง **OSPF** ออกแบบมาให้แบ่งโซนเป็นลำดับชั้นได้ซึ่งเหมาะกับสเกล 500 เราเตอร์มาก ทำให้ประหยัดโหลด
- แต่ กินพลังงาน **CPU** ในการรัน **Dijkstra** เมื่อเกิดการเปลี่ยนแปลงและใช้ **Memory** สูงในการเก็บ **Topology Database**

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

- แม้ **LS** จะกิน **Memory** และ **CPU** มากกว่า **DV** แต่ในปัจจุบัน **router** มี **CPU** และ **RAM** เยอะขึ้นจึงไม่ใช่ข้อจำกัดอีกต่อไป
ด้วยความที่ **scale** เป็นระดับองค์กรจึงต้องมีความเสถียรและความเร็วในการฟื้นตัวซึ่ง **LS** มี **Communication overhead** ในระยะยาว
ที่น้อยกว่าแถมแก้ปัญหาขยายสเกลด้วยการแบ่ง **Hierarchy** ทำให้ **LS** เหมาะใช้ในเครือข่ายองค์กรขนาดใหญ่